UTN FRLP

Desarrollo de Software 2026

Requerimientos y Modelado + TypeScript

Clase #04 - 20 de abril del 2026

Matías Corti - Martina García Améndola

Temas

1º Parte: Requerimientos y Modelado

- Requisitos Funcionales, Requisitos No Funcionales, Restricciones y Reglas de Negocio
- Procedimiento para Armado de Backlog (entregable)

2ª Parte: TypeScript

- De JS a TS: el "por qué"
- Tipos, funciones, interfaces
- Práctica

1º Parte
Requerimientos y Modelado

¿Por Qué es tan Importante?

Un alto % de proyectos fallan o superan presupuesto/tiempo por problemas de requerimientos.

Costo del error: *Arreglar un error de requerimientos en fases tardías es* ***MUCHO*** *más caro.*

Enfoque Profesional: *Entender bien qué se necesita antes de escribir código.*

Desafío Dual:

- *Construir el* ***Producto Correcto*** *(¿Resuelve la necesidad real?) ¡Esto es Análisis de Requerimientos!*
- *Construir el* ***Producto Correctamente*** *(¿Funciona bien, es mantenible?) Esto es Diseño/Implementación.*

¿De qué hablamos cuando hablamos de Requerimientos?

Requerimientos Funcionales

- ¿Qué DEBE HACER el sistema?
- Ejemplos (Sistema de Biblioteca)
  - "Registrar un nuevo libro"
  - "Permitir a socios buscar libros por título".

Requerimientos No Funcionales

- ¿Cómo DEBE SER el sistema?
- Ejemplos
  - Rendimiento (respuesta < 2s)
  - Seguridad (encriptación de datos)
  - Usabilidad (intuitivo)
  - Disponibilidad (99.5% uptime)

Restricciones

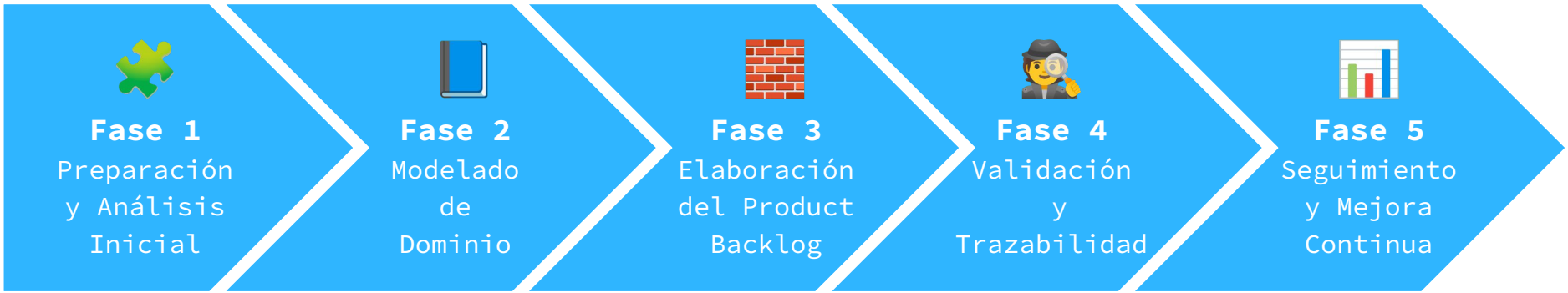
- Son limitaciones: Técnicas, Legales, de Negocio, de Recursos, etc
- Ejemplos
  - Debe usar React para el frontend
  - Debe cumplir con la Ley de Protección de Datos

Requisito Funcional vs Regla de Negocio

Requisito Funcional ≠ Regla de Negocio

	Requisito Funcional (RF)	**Regla de Negocio (RN)**
Pregunta	¿QUÉ hace?	¿Bajo QUÉ CONDICIONES?
Ejemplo	RF-001: "Registrar préstamos"	RN-001: "Máximo 5 préstamos por usuario"
Cambia cuando…	Cambia el alcance	Cambian las políticas

Procedimiento Para Armado de Backlog



LINK AL DOCUMENTO

2ª Parte

TypeScript

Tipado para JavaScript

TypeScript

TIPADO ESTATICO PARA JAVASCRIPT

[TypeScript - Doc]

🔬 SUPERSET DE JAVASCRIPT

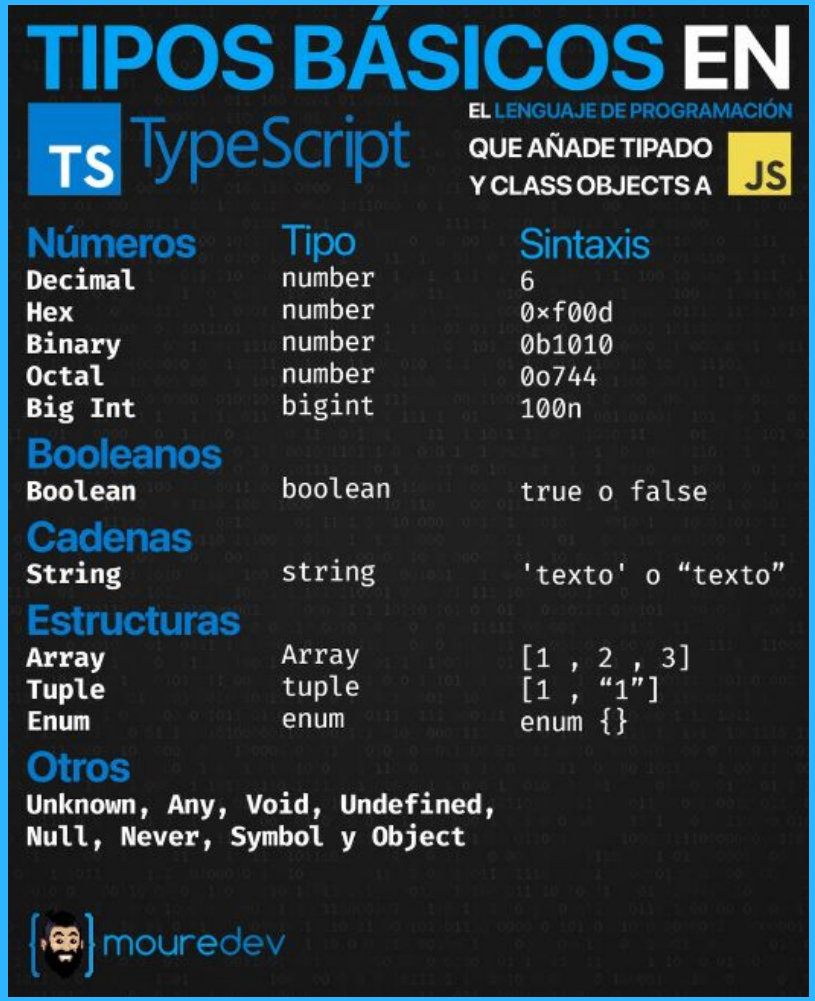
- Transpila a JavaScript puro
- Detección de errores en tiempo de compilación
- **IntelliSense** mejorado en IDEs
- Refactoring seguro a gran escala

⚡ VENTAJAS EN DESARROLLO

- **Type safety** prevención de errores de tipos
- Interfaces y tipos personalizados
- **Generics** para código reutilizable

TypeScript

Tipos Básicos



TypeScript

Funciones tipadas

En Javascript

```javascript
function clasificarNota(nota) {
  if (nota < 4) return "Desaprobado";
  if (nota < 8) return "Aprobado";
  return "Promocionado";
}
```

En TypeScript

```typescript
function clasificarNota(nota: number): string {
  if (nota < 4) return "Desaprobado";
  if (nota < 8) return "Aprobado";
  return "Promocionado";
}

clasificarNota("siete");   // ❌ Error antes de correr
clasificarNota(7);         // ✅ "Aprobado"
```

TypeScript

Clase Básica

Se puede definir una clase usando la palabra clave **class**.

Las clases pueden contener **propiedades** (atributos) y **métodos** (funciones).

```
class Persona {
  nombre: string;
  edad: number;

  constructor(nombre: string, edad: number) {
    this.nombre = nombre;
    this.edad = edad;
  }

  saludar(): string {
    return `Hola, mi nombre es ${this.nombre} y tengo ${this.edad} años.`;
  }
}

const persona1 = new Persona('Juan', 30);
console.log(persona1.saludar()); // Hola, mi nombre es Juan y tengo 30 años.
```

TypeScript

Herencia

Las clases en TypeScript soportan **herencia**, lo que permite que una clase hija herede las propiedades y métodos de una clase base.

```typescript
class Animal {
  nombre: string;

  constructor(nombre: string) {
    this.nombre = nombre;
  }

  mover(): void {
    console.log(`${this.nombre} se está moviendo`);
  }
}

class Perro extends Animal {
  ladrar(): void {
    console.log(`${this.nombre} está ladrando`);
  }
}

const miPerro = new Perro('Fido');
miPerro.mover(); // Fido se está moviendo
miPerro.ladrar(); // Fido está ladrando
```

TypeScript

Interfaces

Una clase puede implementar una **interfaz** para asegurar que siga una estructura determinada.

```typescript
interface Vehiculo {
  marca: string;
  modelo: string;
  moverse(): void;
}

class Moto implements Vehiculo {
  marca: string;
  modelo: string;

  constructor(marca: string, modelo: string) {
    this.marca = marca;
    this.modelo = modelo;
  }

  moverse(): void {
    console.log(`La moto ${this.marca} ${this.modelo} se está moviendo.`);
  }
}

const miMoto = new Moto('Yamaha', 'YZF-R1');
miMoto.moverse(); // La moto Yamaha YZF-R1 se está moviendo.
```

TypeScript

Clases Abstractas

Una **clase abstracta** es una clase que no puede ser instanciada directamente. Debe ser extendida por otras clases. Puede contener métodos abstractos que deben ser implementados por las clases derivadas.

```typescript
abstract class Figura {
  abstract area(): number;

  describir(): void {
    console.log('Soy una figura.');
  }
}

class Cuadrado extends Figura {
  lado: number;

  constructor(lado: number) {
    super();
    this.lado = lado;
  }

  area(): number {
    return this.lado * this.lado;
  }
}

const miCuadrado = new Cuadrado(4);
console.log(miCuadrado.area()); // 16
```

TypeScript

Getters y Setters

Los **getters** y **setters** permiten controlar cómo se accede y modifica una propiedad.

```
class Persona {
  private _edad: number = 0;

  get edad(): number {
    return this._edad;
  }

  set edad(nuevaEdad: number) {
    if (nuevaEdad >= 0) {
      this._edad = nuevaEdad;
    } else {
      console.log('La edad no puede ser negativa.');
    }
  }
}

const persona = new Persona();
persona.edad = 25;
console.log(persona.edad); // 25
persona.edad = -3; // La edad no puede ser negativa.
```



PREPARANDO NUESTRO ENTORNO

Necesitamos:

1️⃣ **Node.js** (runtime de JavaScript)

2️⃣ **NPM** (gestor de paquetes)

```bash
### Parado en el directorio donde irán los proyectos

# Instalación global
npm install -g typescript

# Verificar
tsc --version

# Traspilar archivo .ts -> .js
tsc app.ts        # genera app.js (invocado en HTML)
```

typescriptlang.org



Práctica Javascript



Práctica

Typescript

- 3 ejercicios con dificultad creciente
- Cada ejercicio **en su carpeta** dentro del repositorio:

```
ds2026-actividades/
└── ejercicios/
    └── c04-typecript/
        ├── ej1-medio-arbol/
        ├── ej2-catalogo/
        └── ej3-abm-libros/
```

- Cada ejercicio terminado = **un commit** (con mensaje descriptivo)
- ⏱️ **En clase:** arrancar desde el Ejercicio 1, llegar hasta donde puedan
- 📅 **Durante la semana:** completar hasta el Ejercicio 3

Ejercicios Prácticos

Ejercicio 1 - Medio-árbol de asteriscos TIPADO
Objetivo: *tomar el Ejercicio 5 de C03 (DOM: input y botón) y reconvertirlo a TypeScript para ver la diferencia.*
Pasos:

- Copiar el ejercicio 05 de la clase anterior
- Crear un archivo `app.ts` con las funciones convertidas a typescript.
- Traspilar: `tsc app.ts`
- Abrir en el navegador y probar.
- **Romper algo a propósito**: cambiar la firma de la función `generarAsteriscos()` para que reciba `string` en vez de `number`, y ver cómo `tsc` "se queja" antes de generar el `.js`.

Ejercicio 2 — Catálogo de libros tipado
Objetivo: *practicar interfaces, arrays de objetos, filtros y renderizado en HTML.*
Pasos:

- Crear un index.html con:
  - `<h1>Catálogo</h1>`, `<input type="text" id="filtroAutor">`
  - `<button id="filtrar">Filtrar</button>`
  - `<button id="mostrarDisponibles">Solo disponibles</button>`
  - `<button id="mostrarTodos">Ver todos</button>`
  - `<ul id="listado">` donde se van a mostrar los libros.
  - `<p id="stats"></p>` para mostrar cantidad y precio promedio
  - `<script src="app.js"></script>` al final del `<body>`
  - Crear `app.ts` con una interface **`Libro`** isbn (string), titulo (string), autor (string), precio (number) disponible (boolean) y genero (string/opcional).
- Escribir y tipar estas funciones:
  - `buscarPorAutor(autor: string): Libro[]`
  - `librosDisponibles(): Libro[]`
  - `precioPromedio(libros: Libro[]): number`
  - `renderizar(libros: Libro[]): void` → agrega cada libro como `<li>` en el `<ul>` y actualiza el `<p>` de stats con la cantidad y el promedio.
- Enganchar los botones:
  - Filtrar → leer el input y llamar `renderizar(buscarPorAutor(...))`.
  - Solo disponibles → `renderizar(librosDisponibles())`.
  - Ver todos → `renderizar(catalogo)`.
- Al cargar la página, mostrar todos los libros por defecto.
- Traspilar: `tsc app.ts`
- Abrir en el navegador y probar los tres botones.

Ejercicio 3 - ABM de libros en la página
Objetivo: *ombinar formulario + lista + estado en memoria, todo tipado (Extensión del Ej 2).*
Pasos:

- Partir del HTML del Ej 2 y sumar un formulario para cargar libros (con todos los campos y un `<div id="errorForm"></div>` para mostrar los errores
- Cada `<li>` del listado debe tener además un botón "Eliminar" que lo quita del array y re-renderiza.
- En `app.ts`:
  - Reusar la interface `Libro` del Ej 2.
  - Array `catalogo: Libro[]` que empieza con 2-3 libros precargados.
  - **Funciones tipadas:**
    - `agregarLibro(libro: Libro): void` — lo agrega al array y llama a renderizar.
    - `eliminarLibro(isbn: string): void` — lo saca del array y re-renderiza.
    - `validarFormulario(): Libro | null` — lee los inputs, valida que no estén vacíos / precio > 0, genera un ISBN random (ej: `"AUTO-" + Date.now()`) y devuelve un `Libro`, o `null` si hay error.
  - El handler del botón **Agregar**:
    - Llamar a `validarFormulario()`.
    - Si devuelve `null` → mostrar error en `#errorForm`.
    - Si devuelve un `Libro` → `agregarLibro()` + limpiar el form.
- Traspilar: `tsc app.ts`
- Abrir en el navegador y probar agregar y eliminar libros.

Actividades de la semana

Actividad individual de la semana

1º instancia hasta el **26/04/2026**
2º instancia: ***10/05/2026*** *(Cierre Bloque 1)*

[AI-C04 - Typescript]
Completar Ejercicios Prácticos

- Completar al menos los **ejercicios del 1 al 3**
- Cada ejercicio en su carpeta, cada uno con su commit
- Subir todo al su repositorio ds2026-actividades en GitHub, en un subdirectorio nuevo para esta clase: **ejercicios/c04-typescript/**

Una vez completada la entrega, escribir en Discord un mensaje en el canal **#actividades**, dentro del Hilo: ***"[AI-C04] Typescript".***

📽️ Avanzar con TypeScript

Recomendamos un **tutorial** de dos partes que puede resultarles muy útil:

De Novato a Ninja: Aprende TypeScript - intensivo (#1) [~102 min]

De Novato a Ninja: Aprende TypeScript - intensivo (#2) [~94 min]

Actividad grupal de la semana

Hasta el **26/04/2026** (inclusive)

[AG-C04 - Procedimiento de Backlog: Fases 1 y 2]

Completar Fases 1 y 2 del "Procedimiento Para Armado de Backlog"

1. **Completar Fase 1:** Definir el alcance, módulos y límites (1.1) y Glosario de términos (1.2).
2. **Completar Fase 2:** Modelado de Dominio (2.1), Requisitos funcionales (2.2), Reglas de negocio (2.3) y Diagramas complementarios (2.4)
3. Crear en el repositorio, dentro de la carpeta "Documentación", las carpetas:
   a. "01 - Análisis Inicial"
   b. "02 - Modelado de Dominio"
   Subir en cada una de ellas los documentos y imágenes correspondientes a los recursos creados (anteponiendo números de fase/subfase para mantener el orden).
4. Agregar en la carpeta "Documentación" un archivo README.md inicial con formato markdown, que sirva como índice de los documentos incluídos (ir completando a medida que se suban/linkeen archivos).
5. Comunicar la finalización en Discord en un mensaje único por equipo, dentro del canal **#actividades**, en el Hilo: "***[AG-C04] Procedimiento de Backlog: Fases 1 y 2***".

A partir de ahora, usen **PRs (Pull requests)** para repartirse la subida de elementos y cambios entre los miembros del equipo.